

Chapter 3

Predictive Encoding

In the previous chapter we saw that most pictures can be characterized by high spatial and temporal picture correlation. All pictures exhibit nonuniform pixel distribution to a certain degree and signal processing can greatly increase this nonuniformity. If we can exploit these characteristics, we can reduce the amount of information that needs to be transmitted to adequately represent a picture. In the next two chapters, the video coding tools that exploit these characteristics are discussed. This chapter covers predictive encoding techniques including motion-compensated prediction. Chapter 4 looks at transform encoding. Together, predictive encoding and transform encoding form the heart of the MPEG-2 video encoder.

3.1. ENTROPY CODING

The idea of using variable length coding to efficiently represent messages is known as entropy coding. If we are to be able to make regular use of it, we need some automatic method for calculating the appropriate variable word length codes dictated by a particular set of symbol probabilities. The best and most widely used technique is called Huffman coding. The current video and audio compression standards used for digital television include variable length code words generated as Huffman codes. We will now study how to generate these useful code word sets.

3.1.1. Huffman Coding

The steps involved in generating a set of Huffman code words are summarized below.

- Step 1:** List the symbols to be transmitted in decreasing order of probability.
- Step 2:** Combine the two symbols with the smallest probabilities and reorder the symbols in decreasing order of probability.
- Step 3:** Repeat Step 2 until only two combined symbols remain.

Step 4: Mark each combination consistently with either

$$\left. \begin{array}{l} 1 \\ 0 \end{array} \right] \quad \text{or} \quad \left. \begin{array}{l} 0 \\ 1 \end{array} \right]$$

Step 5: Read off the Huffman code words from right to left.

The entire process is best illustrated by way of an example.

EXAMPLE 3.1

The symbols a, b, c, and d have probabilities 0.75, 0.125, 0.0625, and 0.0625, respectively. Calculate the Huffman code words appropriate for these symbols.

SOLUTION We begin by listing the code words in decreasing order of probability (Step 1) as shown in Figure 3.1.

a	0.75
b	0.125
c	0.0625
d	0.0625

Figure 3.1 Initial ordering of symbol probabilities.

Now we combine the two symbols with the smallest probabilities and reorder in decreasing order of probability (Step 2). In this case after combining, the two smallest probabilities are equal. It is not important to the final outcome that one is placed higher in the reordered list. In this case we have placed the newly merged symbols last as shown in Figure 3.2. We will return at the end of the example to look at the impact of the other choice.

a	0.75	-----	0.75
b	0.125	-----	0.125
c	0.0625	]----->	0.125
d	0.0625		

Figure 3.2 Probabilities after first combining and reordering.

We now again combine the two smallest probabilities and reorder (Step 2). The result is shown in Figure 3.3. As this leaves us with only two probabilities, we can stop the combining process (Step 3).

a	0.75	-----	0.75	-----	0.75		
b	0.125	-----	0.125	]----->	0.25		
c	0.0625	]----->	0.125				
d	0.0625						

Figure 3.3 Probabilities after second combining and reordering.

This is called the Huffman tree. We now proceed to mark each combination operation (Step 4). This is shown in Figure 3.4.

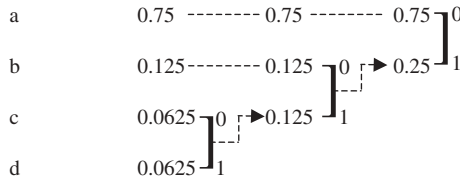


Figure 3.4 The marking of each combination operation.

The Huffman code word for each symbol can be read back from right to left in the Huffman tree as shown in Figures 3.5–3.8.

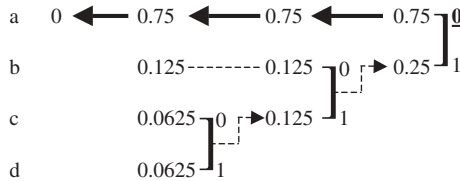


Figure 3.5 Reading off the Huffman code word for symbol a.

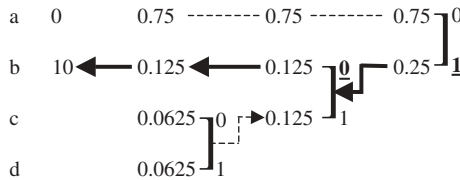


Figure 3.6 Reading off the Huffman code word for symbol b.

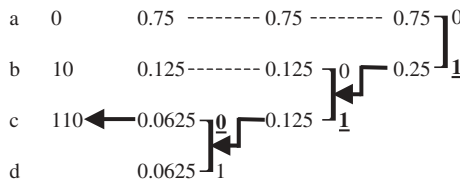


Figure 3.7 Reading off the Huffman code word for symbol c.

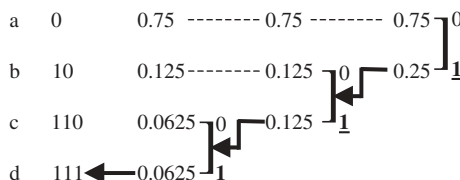


Figure 3.8 Reading off the Huffman code word for symbol d.

Table 3.1 Huffman code words for Example 3.1.

Symbol	Huffman code word
a	0
b	10
c	110
d	111

Thus the Huffman code words for this example are as shown in Table 3.1.

No short code word ever forms a prefix for a longer code word in a Huffman code. This means that there is no need for any form of space between code words, and so any bit stream is uniquely decipherable.

Had we reordered the other way after the first combination operation, the final Huffman tree would be as shown in Figure 3.9.

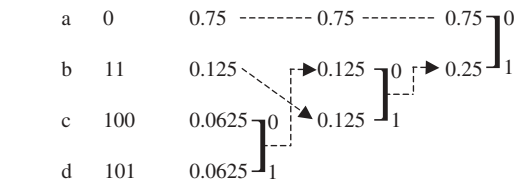


Figure 3.9 Huffman tree if alternate reordering after first symbol combination.

In this case, the Huffman code words are as shown in Table 3.2.

Table 3.2 Huffman code words for alternate ordering in Example 3.1

Symbol	Huffman code word
a	0
b	11
c	100
d	101

The code words for symbols b, c, and d have changed. However, each code word is the same length as in the previous Huffman code calculation in this example. The average code word length in each case would therefore be identical.

We now look at a simple example of coding and decoding symbols using Huffman codes. ■

EXAMPLE 3.2

Use the Huffman code defined in Table 3.1 to encode the stream of symbols shown in Figure 3.10.

Figure 3.10 Original stream of symbols for Example 3.2.

After the encoding process, we have the code words shown in Figure 3.11.

0 10 110 111 0 0 0 10

Figure 3.11 Symbols from Figure 3.10 encoded using Huffman code from Table 3.1.

This can be concatenated to form the final transmitted bit stream shown in Figure 3.12.

01011011100010

Figure 3.12 Final transmitted bit stream.

The decoding process consists of reading values from the bit stream until a valid code word is detected. Because no Huffman code word ever forms the prefix of a longer Huffman code word, this always results in correct decoding in an error-free environment. The decoding process is shown in Figure 3.13.

0	1011011100010							
a								
0	1	011011100010						
a	?							
0	10	11011100010						
a	b							
0	10	1	1011100010					
a	b	?						
0	10	11	011100010					
a	b	?						
0	10	110	11100010					
a	b	c						
0	10	110	1	1100010				
a	b	c	?					
0	10	110	11	100010				
a	b	c	?					
0	10	110	111	00010				
a	b	c	d					
0	10	110	111	0	0010			
a	b	c	d	a				
0	10	110	111	0	0	010		
a	b	c	d	a	a			
0	10	110	111	0	0	0	10	
a	b	c	d	a	a	a		
0	10	110	111	0	0	0	1	0
a	b	c	d	a	a	a	?	
0	10	110	111	0	0	0	10	
a	b	c	d	a	a	a	b	

Figure 3.13 Huffman decoding process. ■

The introduction of transmission errors can have a significant effect on the decoding process. This is illustrated in Example 3.3.

EXAMPLE 3.3

The bit stream generated in Example 3.2 is received with a single bit error in the second bit of the bit stream. Determine the decoded sequence in this case.

SOLUTION The received bit stream is given in Figure 3.14.

00011011100010

Figure 3.14 Bit stream generated in Example 3.2 with a single bit error.

The decoding process is shown in Figure 3.15.

0	0011011100010								
a									
0	0	011011100010							
a	a								
0	0	0	11011100010						
a	a	a							
0	0	0	1	1011100010					
a	a	a	?						
0	0	0	11	011100010					
a	a	a	?						
0	0	0	110	11100010					
a	a	a	c						
0	0	0	110	1	1100010				
a	a	a	c	?					
0	0	0	110	11	100010				
a	a	a	c	?					
0	0	0	110	111	00010				
a	a	a	c	d					
0	0	0	110	111	0	0010			
a	a	a	c	d	a				
0	0	0	110	111	0	0	010		
a	a	a	c	d	a	a			
0	0	0	110	111	0	0	0	10	
a	a	a	c	d	a	a	a		
0	0	0	110	111	0	0	0	1	0
a	a	a	c	d	a	a	a	?	
0	0	0	110	111	0	0	0	10	
a	a	a	c	d	a	a	a	b	

Figure 3.15 Huffman decoding process for the errored bit stream of Figure 3.14.

Comparing the decoded sequence for the errored bit stream with the correctly decoded sequence, we see that the second and third symbols (b and c) have been incorrectly received (as a and a) and an additional symbol (c) is also decoded before correct decoding resumes with the next symbol (d). The compression achieved using Huffman coding is achieved at the expense of greater vulnerability of the generated bit stream to errors. Modern video coders are designed to minimize the effect of this vulnerability. However, it cannot be completely removed. ■

Another limitation of Huffman code words is the requirement that each code word is an integer number of bits in length. If the probability of a symbol is $1/3$, the optimum number of bits to assign is 1.6. The Huffman code assigns either one or two bits—either choice leads to a longer compressed message than is theoretically necessary. The problem is particularly serious when the probability of one symbol is very high. For example, if the probability of one symbol is 0.9 then the optimal code word length would be 0.15 bits. A Huffman code would assign a one bit code word. This is the reason that Huffman codes cannot exactly achieve the entropy of the symbol stream. This problem can be addressed by using a different scheme known as arithmetic coding. Although arithmetic coding features in standards such as MPEG-4 and JPEG-2000, it is not a part of the MPEG-2 standards that form the basis of digital television broadcasting services. It is therefore not discussed further here.

3.1.2. Run Length Coding

In run length coding, a run of consecutive identical symbols is combined together and represented by a single variable length (e.g., Huffman) code word. A simple example is the transmission of a black-and-white facsimile. Consider the line segment shown in Figure 3.16.

The message to be transmitted would be

(3 white pixels) (2 black pixels) (5 white pixels) (1 black pixel) (2 white pixels)
(6 black pixels) (3 white pixels) (3 black pixels) (3 white pixels) ...

If the statistics of the various run lengths are calculated, appropriate Huffman code words can be designed. In the case of facsimile transmission, separate Huffman code word sets have been designed for runs of white pixels and runs of black pixels. This design strategy has been used because the statistics for the different types of runs are not the same. In typical documents, long runs of white pixels are more common than long runs of black pixels.

3.2 PREDICTIVE CODING

In Chapter 2, we have seen that knowing the value of a particular pixel provides a considerable amount of information about the pixels that surround it due to the



Figure 3.16 Segment of a black-and-white facsimile.

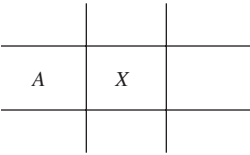


Figure 3.17 Simple one-dimensional prediction.

high correlation that typically exists between nearby pixels. What is needed is some way to exploit this correlation so that the amount of information that needs to be transmitted can be reduced. One way of achieving this is by a technique known as predictive encoding.

The aim of predictive encoding is to use the values of already transmitted pixels to predict the value of the current pixel to be transmitted. It is then only necessary to transmit the difference between the prediction and the actual pixel value. If the prediction is accurate, this is usually a small value and so less bits of data will need to be transmitted in order to represent it.

The simplest form of predictive encoding is to predict the value of the pixel to be transmitted using the value of the pixel immediately to its left. This is illustrated in Figure 3.17. This approach is often called one-dimensional *differential pulse code modulation* (DPCM). The difference between the prediction and the actual pixel value is then transmitted.

In Figure 3.17, X is the pixel to be transmitted and A is the value of the pixel immediately to its left. We estimate the value of X as A that is

$$\hat{X} = A$$

The value transmitted to the receiver is then

$$\text{Difference} = X - \hat{X}$$

$$\text{Difference} = X - A$$

EXAMPLE 3.4

Consider the set of pixel values given in Figure 3.18. Calculate the values of the prediction difference when one-dimensional DPCM is employed. We assume that the first pixel in a line is predicted by the mid-gray value 128.

130	135	141	129	151
124	140	165	200	199
119	132	175	205	203

Figure 3.18 Pixel values for Example 3.4.

The pixel values are repeated in Figure 3.19 with the prediction value for each pixel in italics directly below it. Thus the first pixel in the first row (130) is predicted by mid-gray value (128), the second value in the first row (135) is predicted by the first value in the first row (130), and so on.

Pixel value	130	135	141	129	151
Prediction	128	130	135	141	129
Pixel value	124	140	165	200	199
Prediction	128	124	140	165	200
Pixel value	119	132	175	205	203
Prediction	128	119	132	175	205

Figure 3.19 Pixel values with appropriate predictor immediately below the pixel value.

The prediction difference is then simply calculated by subtracting the prediction from the true pixel value giving the result shown in Figure 3.20.

+2	+5	+6	-12	+22
-4	+16	+25	+35	-1
-9	+13	+43	+30	-2

Figure 3.20 Prediction difference for Example 3.4. ■

The following MATLAB example applies one-dimensional DPCM to the first picture in the “Mobile and Calendar” sequence.

EXAMPLE 3.5—MATLAB

Calculate the one-dimensional DPCM prediction difference picture for the first picture in the “Mobile and Calendar” sequence. Also calculate the entropy of this prediction difference picture.

SOLUTION The process is simply that the prediction difference for column i is the pixel value in column i minus the pixel value in column $(i - 1)$. The exception is the first column where there is no pixel to the left from which to form a prediction. This column is predicted with the mid-gray value 128. If the prediction difference is stored in an array DIFF then appropriate MATLAB code is given below. It is assumed that the an array `calendar_1` contains the luminance pixel values of the first frame of the sequence “Mobile and Calendar.”

```

[row,col] = size(calendar_1);
DIFF = zeros(row,col);
DIFF(:,1) = calendar_1(:,1) - 128*ones(row,1);
for col_number = 2:col
    DIFF(:,col_number) = calendar_1(:,col_number) - calendar_1(:,col_number-1);
end

```

The histogram for the prediction difference contained in an array DIFF is shown in Figure 3.21.

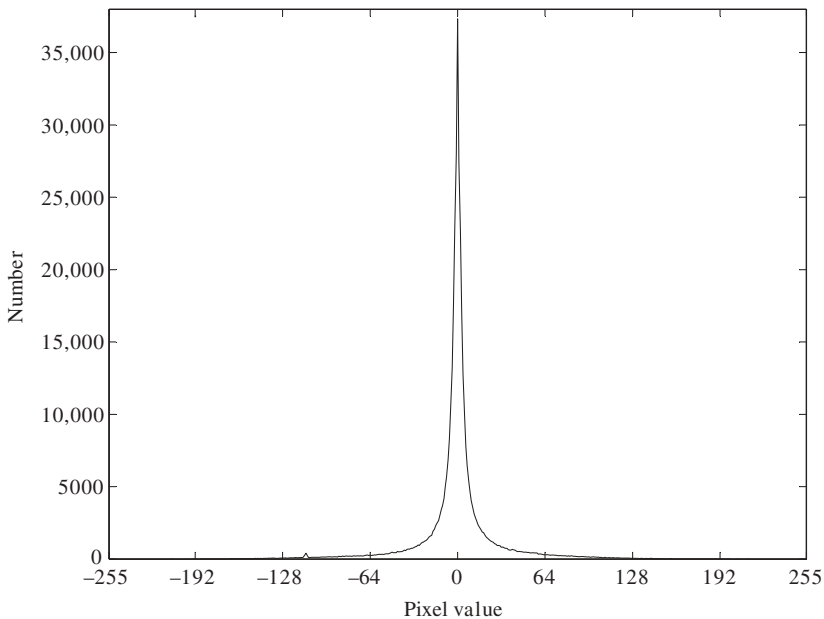


Figure 3.21 Histogram of one-dimensional DPCM prediction difference.

As we had hoped, Figure 3.21 shows that the prediction difference is usually small with zero, the most common prediction difference and almost all prediction differences in the range of ± 64 . The possible range of pixel values has increased from 0 to 255 in the original picture to -255 to $+255$ in the prediction difference. If the prediction was not reasonably accurate, we could actually have an increase in the average number of bits needed to represent each pixel. The entropy for the prediction difference is 5.95 bits/pixel. This represents a small saving (just over 20%) on the original picture entropy of 7.61 bits/pixel. Although this is far less than is needed for the introduction of digital television services, it has been achieved in a lossless manner.

Figure 3.22 shows the prediction difference picture for one-dimensional DPCM (after shifting by 128 so that mid-gray value represents 0 and scaling by a factor of 2 to make the prediction difference clearer).

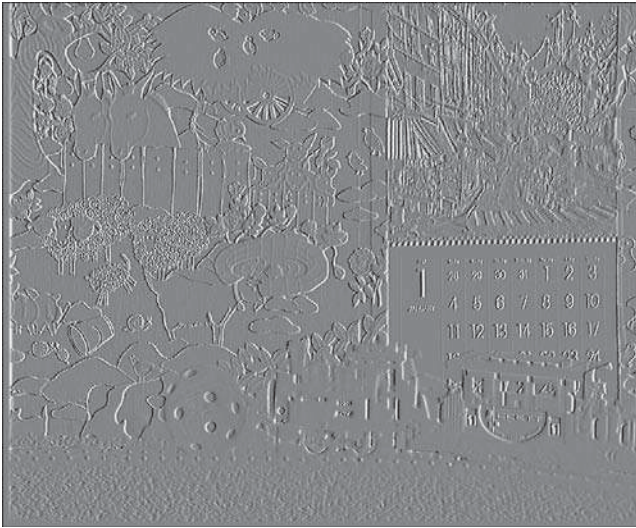


Figure 3.22 One-dimensional DPCM difference picture for first picture in “Mobile and Calendar” sequence (shifted by 128 and scaled by 2).

It is apparent that most of the large prediction differences occur at the edges of objects within the picture. ■

In the previous example, the pixel to the left of the pixel to be transmitted was used as the predictor. There is, of course, no reason why this pixel needs to be the one used. Any pixel whose value has already been transmitted to the receiver (so that the receiver can form an identical prediction) can be used. For example, the pixel immediately above (i.e., on the previous line to) the pixel to be transmitted could be used to form the prediction. Indeed, there is no need for the prediction to be based on just a single pixel. Figure 3.23 shows a generalized two-dimensional predictor.

In this case, the prediction of X ($\hat{X}$) can be formed according to

$$\hat{X} = k_1A + k_2B + k_3C + k_4D$$

The values of the weighting factors (k_i) are picture-dependant. Using pixels both to the left and above the pixel to be predicted means that both the horizontal and vertical correlations within the picture are exploited. Some improvement in performance would therefore be expected.

B	C	D
A	X	

Figure 3.23 Two-dimensional prediction.

EXAMPLE 3.6—MATLAB

Implement a two-dimensional predictor of the form given below.

$$\hat{X} = 0.5A + 0.5C$$

Calculate the entropy of the prediction difference using MATLAB for the first picture in the “Mobile and Calendar” sequence.

SOLUTION First we need to form the prediction. For pixels in the first row of the picture, we assume that the pixel above the pixel to be predicted has value 128 (mid-gray). For pixels in the first column of the picture, we assume that the pixel to the left of the pixel to be predicted has value 128. The prediction and the prediction difference can then be formed according to the MATLAB code given below. As before, the array `calendar_1` contains the luminance pixel values of the first frame of the sequence “Mobile and Calendar.”

```
[row,col] = size(calendar_1);
Guess = zeros(row,col);
for irow = 1:row
    if(irow == 1)
        Guess(irow,:) = Guess(irow,:) 1 0.5*128*ones(1,col);
    else
        Guess(irow,:) = Guess(irow,:) + 0.5* calendar_1(irow-1,:);
    end
end
for icol = 1:col
    if(icol == 1)
        Guess(:,icol) = Guess(:,icol) + 0.5*128*ones(row,1);
    else
        Guess(:,icol) = Guess(:,icol) + 0.5* calendar_1(:,icol-1);
    end
end
PRED = A - fix(Guess);
```

The picture entropy is 5.79 bits/pixel in this case, a reduction of 0.16 bits/pixel (2.5%) compared to one-dimensional prediction. This is a modest improvement given the additional computational requirements, but has still been achieved in a lossless manner. ■

Researchers have shown that once pixels A and C in Figure 3.23 have been used, there is little to be gained by using additional pixels in the same picture. Predictive encoders where all of the pixels used to form the prediction are in the same picture as the pixel to be predicted are called intrapicture predictive encoders.

As well as predicting from other pixels within the current picture, it is also possible to predict from a previous picture or pictures. If there has been little motion between the pictures then the prediction can be efficient. This is called interpicture prediction.

EXAMPLE 3.7—MATLAB

Use the first picture of the sequence “Mobile and Calendar” to predict the second picture in the sequence. Display the prediction difference picture and calculate its entropy.

SOLUTION The appropriate MATLAB code is given below. It is assumed that `calendar_1` and `calendar_2` contains the first and second frames, respectively, of the sequence “Mobile and Calendar.”

```
DIFFERENCE = calendar_2 - calendar_1;
```

The resulting prediction difference picture (after shifting by 128 so that mid-gray represents 0 and scaling by a factor of 2 to make the prediction difference clearer scaled by a factor of 2) is shown in Figure 3.24.



Figure 3.24 Interpicture prediction difference picture (shifted by 128 and scaled by 2).

The entropy of the difference picture is 6.18 bits/pixel. This is higher than either the one-dimensional or the two-dimensional intrapicture predictors, meaning that the prediction in this case requires more information to be transmitted when compared to the intrapicture predictors. ■

It might initially seem surprising that the interpicture prediction performs worse than the intrapicture prediction. The first thing to note is that the sequence “Mobile and Calendar” from which these pictures are extracted contains a large amount of motion. MATLAB provides a convenient means of viewing this motion using the `movie` function, and this is illustrated in Example 3.8.

EXAMPLE 3.8—MATLAB

Display as a sequence the first five pictures of the sequence “Mobile and Calendar.” Assume that the first five pictures are held in arrays `calendar_1` to `calendar_5`, respectively.

SOLUTION The MATLAB code to achieve this is given below.

```
display_picture(calendar_1);
M(:,1) = getpicture;
display_picture(calendar_2);
M(:,2) = getpicture;
display_picture(calendar_3);
M(:,3) = getpicture;
display_picture(calendar_4);
M(:,4) = getpicture;
display_picture(calendar_5);
M(:,5) = getpicture;
movie(M,-50,25);
```

Each picture needs to be displayed prior to being stored into the movie array M . This is achieved by the function `display_picture`. The MATLAB code for this function is given below.

```
function display_picture(CURR)
image(CURR);
colormap(gray(256))
set(gca,'XTick',[])
set(gca,'YTick',[])
imHandle=gco;
imageWidth = size(CURR,2);
imageHeight = size(CURR,1);
set(gca,'Units','pixels');
set(gcf,'Units','pixels');
figPos=[10,10,imageWidth,imageHeight];
axPos=[1,1,imageWidth,imageHeight];
set(gcf,'Position',figPos);
set(gca,'Position',axPos);
pause(1);
```

The final movie command causes the pictures stored in the array M to be played forward and backward 50 times at 25 pictures/s. Using `+50` for the second variable in the function call causes the picture to be played in the forward direction only. In this case, the jerkiness between picture 5 at the end of a forward play and picture 1 at the start of the next forward play is subjectively distracting.

Looking at the sequence we see that the camera is panning continuously to the left. At the same time, the train is moving from right to left pushing the ball while the calendar is moving up. Almost every part of the picture is in motion, which explains why interpicture prediction is not especially successful in this case. ■

The impact of motion can be clearly seen if we compare a pair of pictures separated by more than one picture time. Figure 3.25 shows the prediction difference picture (shifted and scaled by 2) when the first picture of the sequence “Mobile and Calendar” is used to predict the fourth picture of the sequence.

The impact of the motion is clearly apparent, as many objects appear more than once in the prediction difference picture (e.g., spots on the ball, numbers on the calendar). This is because we need to put the object in its new position in the current picture as well as to remove it from its former position in the prediction

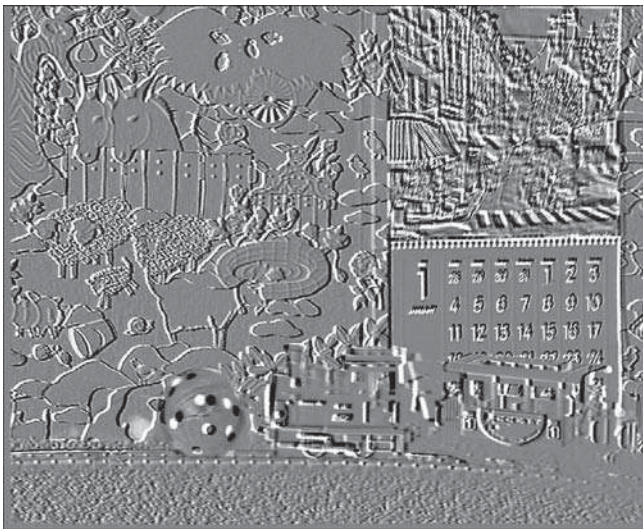


Figure 3.25 Interpicture prediction difference when pictures are separated by three picture times (shifted by 128 and scaled by 2).

picture. These objects thus contribute twice to the entropy of the prediction difference picture.

The simple interpicture predictor described above uses the pixel in the same location in the previous picture to predict the pixel in the current picture. This could be easily extended to include pixels in both the current and the prediction picture. Such a predictor is known as a three-dimensional predictor and is illustrated in Figure 3.26.

In this case the predictor would be

$$\hat{X}=k_1A+k_2B+k_3C$$

Such predictors are rarely used in practice as temporal prediction is invariably combined with motion compensation. Motion-compensated prediction is covered in the next section of this chapter. Another technique, known as transform coding, is used to code the prediction difference after this motion-compensated temporal prediction. Transform coding is the topic of Chapter 4.

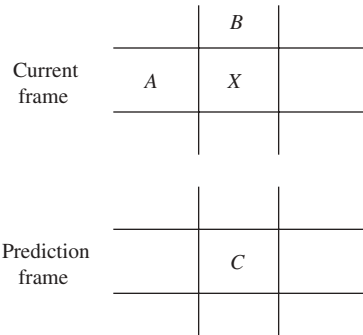


Figure 3.26 Three-dimensional prediction.

3.3 MOTION-COMPENSATED PREDICTION

The performance of interpicture prediction would be greatly improved if we were somehow able to take account of the motion that occurs between pictures. Success would mean that instead of moving objects needing to be coded twice in a picture, we may be able to remove the requirement to code them at all. Instead, information about how the object had moved between pictures would be transmitted.

Although this sounds fine in theory, the estimation of motion in a sequence of pictures is complex. Even if we can assume that the object is rigid (i.e., does not change shape), motion can be made up of several components such as

- zoom (i.e., change of the camera focal length)
- pan (i.e., rotation around an axis normal to the camera axis)
- rotation around the camera axis
- translation along the camera axis
- translation in the plane normal to the camera axis.

Each of these motion types affects the temporal appearance of the resulting video sequence in a particular way. Indeed, complex motion models have been proposed in an attempt to accurately take account of all of these various types of motion. Attempting to implement these models in real time hardware would be impossibly complex. Fortunately approximating all motion as translational motion in a plane normal to the camera axis is a satisfactory approach provided that this is done on sufficiently small objects.

The first part of the process is to attempt to estimate the motion of the object. Motion estimation is performed by searching the prediction picture for the best match to the object of interest in the current picture to be coded. Let us consider an example where rotational motion is estimated using only translation.

EXAMPLE 3.9—MATLAB

The picture in Figure 3.27 is a rotated version of the picture in Figure 3.28. Each picture has size 256×256 pixels. The picture is broken into 16×16 pixel blocks for motion estimation purposes. Calculate the motion-compensated prediction of Figure 3.27 using the picture in Figure 3.28 as the basis for the prediction.

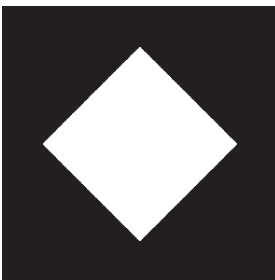


Figure 3.27 Current picture.

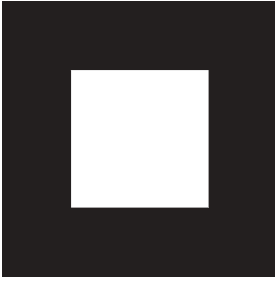


Figure 3.28 Prediction picture.

SOLUTION The motion-compensated prediction picture is shown in Figure 3.29. Using only translational motion to estimate an object that has undergone rotational motion leads to

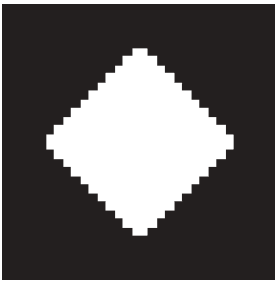


Figure 3.29 Motion-compensated prediction.

a reasonable approximation of the real motion in this case. The motion compensated prediction difference and the interpicture difference are shown in Figure 3.30. The improvement

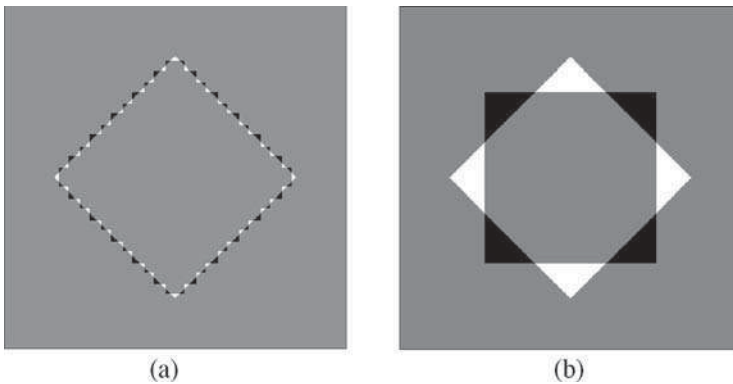


Figure 3.30 (a) Motion-compensated prediction difference. (b) Interpicture difference.

as a result of the motion compensation process is clearly apparent. In fact, all of the types of motion listed earlier can usually be approximated by translational motion providing that the block size is sufficiently small. ■

3.3.1. Motion Estimation

The previous example demonstrates that translational motion can be used to form a reasonable estimate to rotational motion. However, we still need to develop a technique by which this translational motion can be estimated.

One approach is to partition each picture in the video sequence into its individual objects. A previous prediction picture can then be searched for the same object and the motion of the object between pictures can be estimated. Although this may provide excellent performance, there are a number of difficulties associated with this approach. The first of these is difficulty of finding individual objects within a picture. This process, known as segmentation, has been studied for a long time. Despite this, reliable segmentation techniques have not been found. The second problem is that objects do not move in a purely translational manner. It is therefore highly likely that there would be differences between the object as it appears in the current picture and how it appears in the prediction picture making the matching process far more difficult.

The usual approach to motion estimation is to divide the current picture to be predicted into a number of blocks. The prediction picture, which has already been coded, is then searched for the best matching block to each of these blocks in the current picture. Example 3.10 demonstrates the process.

EXAMPLE 3.10—MATLAB

Use picture 1 of the “Mobile and Calendar” sequence (the prediction picture) to predict picture 5 of the same sequence (the current picture) using motion-compensated prediction. Use a block size of 144×176 pixels.

SOLUTION The two pictures are shown in Figure 3.31. The spacing between the pictures has been chosen to allow the motion between pictures to be more easily seen.

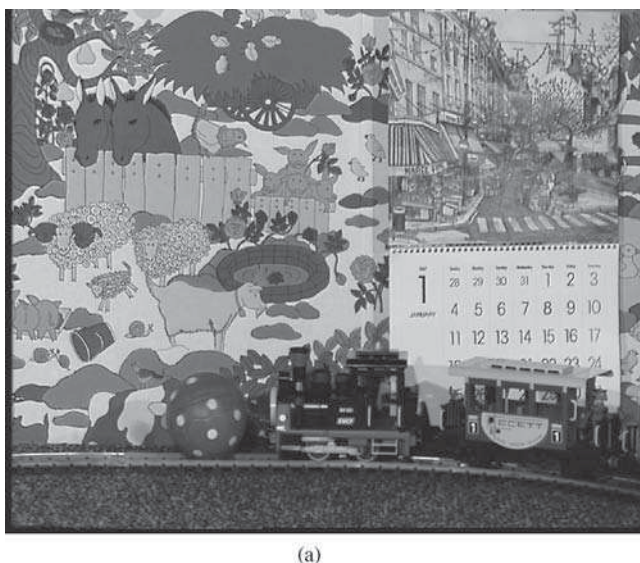


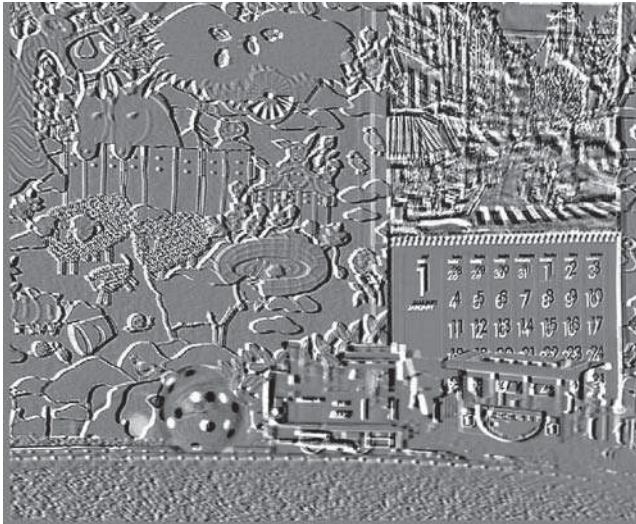
Figure 3.31 (a) Prediction picture: first picture in “Mobile and Calendar” sequence. (b) Current picture: fifth picture in “Mobile and Calendar” sequence.



(b)

Figure 3.31 (Continued)

The picture difference is displayed in Figure 3.32 after shifting by 128 and scaling by 2.

**Figure 3.32** Picture difference for the pictures shown in Figure 3.31 (shifted by 128 and scaled by 2).

The motion that has occurred between pictures is clearly evident in the difference picture especially in the spots on the rolling ball and the numbers on the calendar. The aim of motion estimation is to find a block in the prediction picture (picture 1) that best matches each block in the current picture (picture 5). Figure 3.33(a) shows the current picture divided

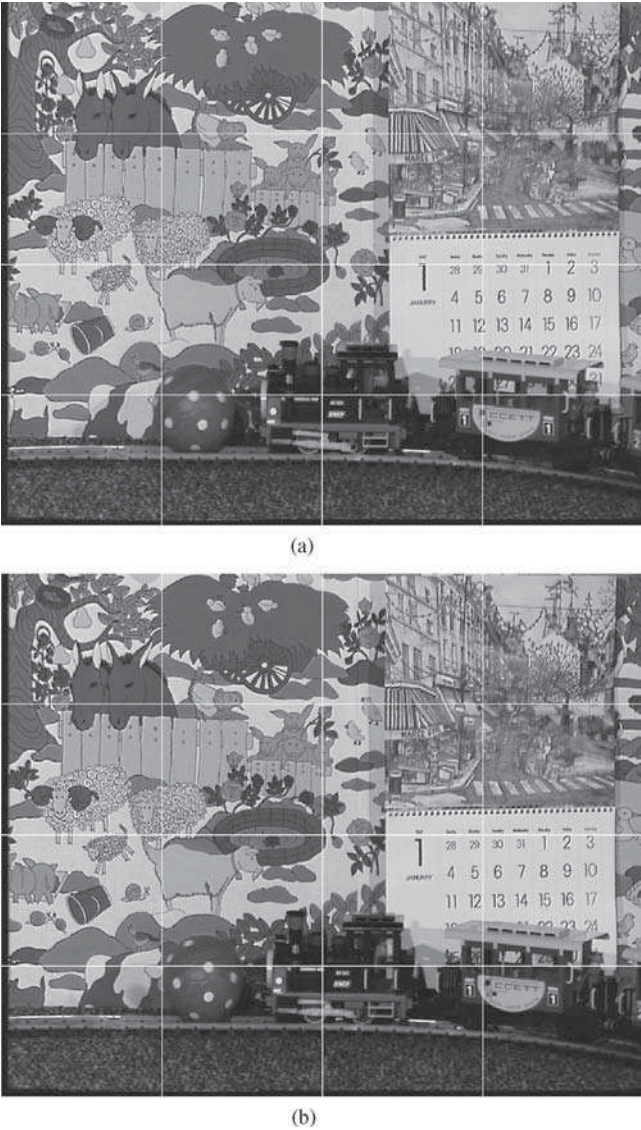


Figure 3.33 (a) Current picture (picture 5) divided into blocks. (b) Prediction picture (picture 1) divided into blocks.

into 144×176 pixel blocks whereas Figure 3.33(b) shows the prediction picture divided into identical blocks. The grid overlay highlights the motion that has occurred between pictures.

We now need to find the block from the prediction picture that best matches each block in the current picture. For simplicity, we consider just one of the blocks so formed—the third block in the third row. This block contains a section of the background wallpaper, part of the train, and a section of the calendar including the large number one. Comparing this block

with the block in the same position in the prediction picture, the prediction difference is as shown in Figure 3.34. The motion between the pictures is again clearly evident.



Figure 3.34 Interpicture difference for block of interest (shifted by 128 and scaled by 2).

The motion estimation process consists of finding the same sized block in the prediction picture that best matches the block in the current picture. One simple method of calculation would indicate that the best matching block would be found four pixels to the left and one pixel up from the colocated block in the prediction picture. This block is highlighted in the prediction picture in Figure 3.35.



Figure 3.35 Best matching block in the prediction picture.

The absolute difference between the block in the current picture and the displaced block from the prediction picture is shown in Figure 3.36. This is called the motion-compensated difference.

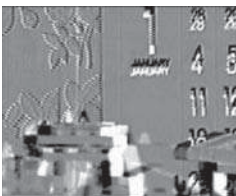


Figure 3.36 Motion-compensated prediction difference (shifted and scaled by 2).

The improvement when the result shown in Figure 3.36 is compared with the inter-picture difference shown in Figure 3.34 is clearly evident. The mean absolute difference is reduced from 25.5 to 18.4 (a reduction of 30%). Note that though the background wallpaper is quite well predicted, the numbers on the calendar and the train are much less well predicted. This is a result of the fact that the motion of the each of these objects is different. The motion estimation approach employed does not allow for two or more different motions in the same block to be estimated. ■

Motion-compensated prediction consists then of two parts: motion estimation (which is performed at the encoder only) and motion compensation (which needs to be performed both at the encoder and at the decoder).

The motion estimation process consists of dividing the current picture to be encoded into a number of blocks. The prediction picture is then searched for the best match to each of the blocks in the current picture. The process can be best thought of as involving three pictures—the current picture, the prediction picture, and the motion-compensated prediction, as illustrated in Figure 3.37. Both the current and the prediction picture are composed of two round objects. The position of each object has moved between pictures. The motion-compensated prediction is initially blank.

Each block in the current picture is considered in turn. For convenience, we will start with the block that contains the left-hand round object. We define a search area in the prediction picture and then look for the block of pixels that best matches the pixels in the selected block in the current picture. This is illustrated in Figure 3.38 with the search area indicated by the highlighted square. The size of the search area is user selectable. The larger the search area, the larger the motion that can be tracked and hence the more likely a good match. However, this comes at the expense of higher computation requirements. We will consider this topic in more detail a little later.

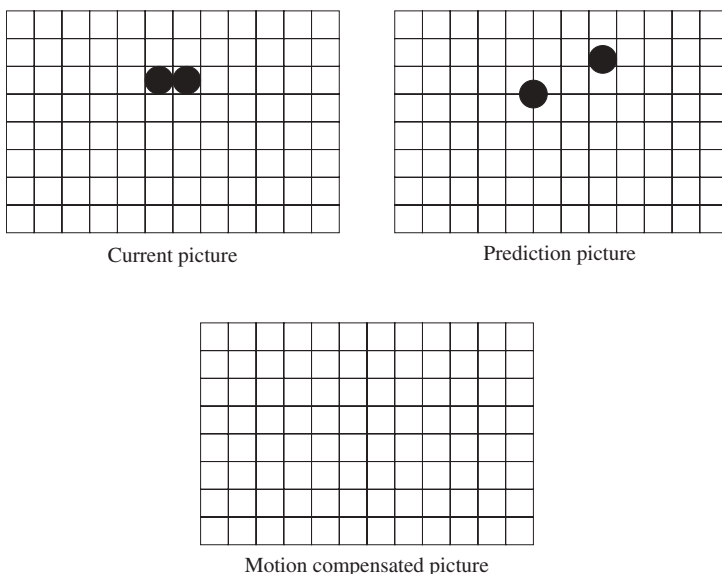


Figure 3.37 Current picture, prediction picture, and motion-compensated prediction.

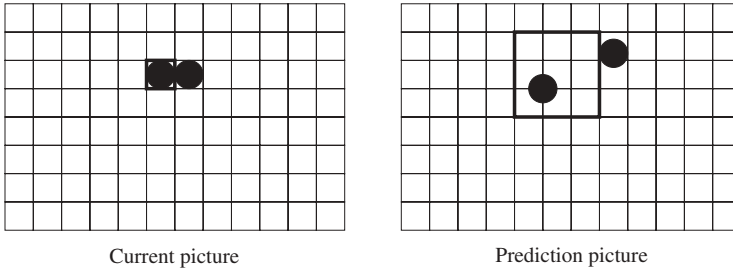


Figure 3.38 Search region in the prediction picture for the selected block in the current picture.

The best matching block¹ in the prediction picture is the one indicated in Figure 3.39. This block is then copied into the same position in the motion-compensated prediction as the block to be predicted in the current picture. This is also illustrated in Figure 3.39.

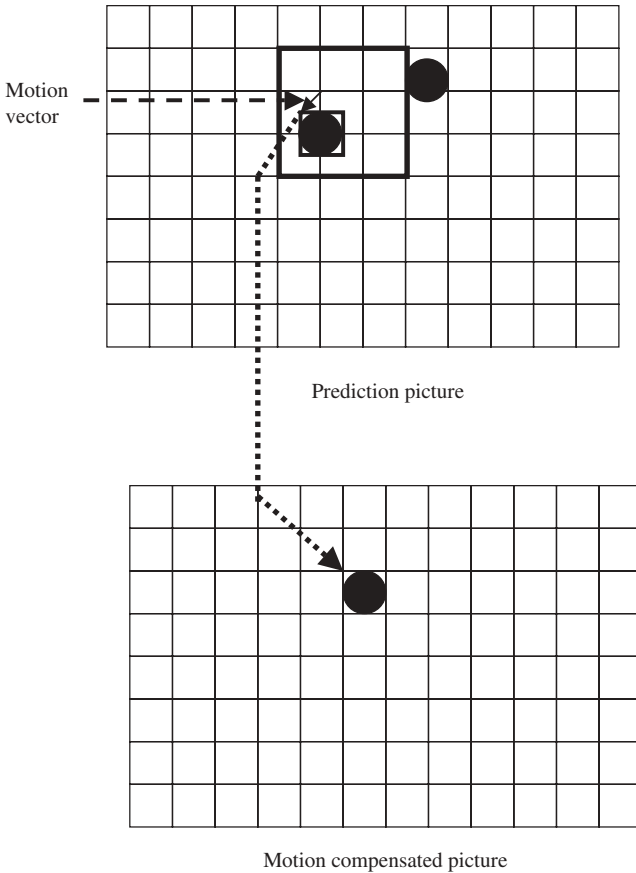


Figure 3.39 Generation of block in motion-compensated prediction.

¹The best matching block in this case is obvious. For a more general pair of pictures, we need some matching criteria that can be used to select the best match. This is covered a little later.

The location of the block in the prediction picture used to predict the selected block in the current picture is transmitted to the decoder because the decoder does not have access to the same information as the encoder (e.g., original versions of the current and prediction pictures) and so cannot calculate the location itself. The information transmitted to the decoder is called a motion vector. The motion vector in this case is shown in the prediction picture in Figure 3.39 by a solid arrow. It marks the displacement from the location of the top left-hand corner of the block in the current picture that is being predicted to the location of the top left-hand corner of the chosen prediction block in the prediction picture. The arrow in Figure 3.39(a) indicates that the block to be used is several pixels to the left and below the location of the block in the same position as the block being predicted in the current picture.

This process is repeated for the next block in the current picture, and this is shown in Figure 3.40.

In this highly idealized case, the motion compensated prediction is identical to the original picture. Thus the prediction difference when the current picture is predicted using the motion-compensated prediction would be a picture consisting entirely of zeros. The only information that would need to be transmitted to the decoder is the motion vectors.

The search area could of course be made larger. An example of this is shown in Figure 3.41 for the case of the block containing the left-hand object. There are now two possible best match blocks. Each is shown with its corresponding motion vector

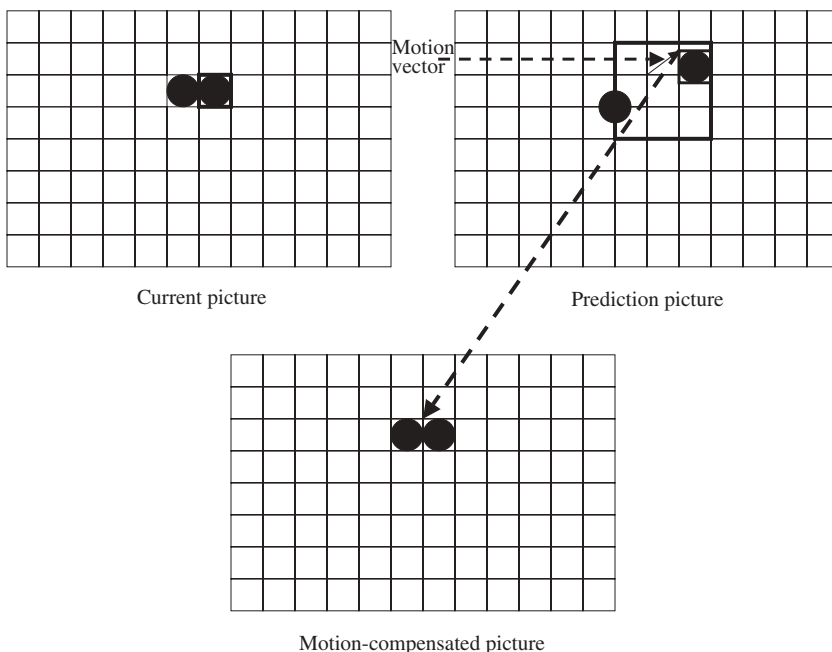
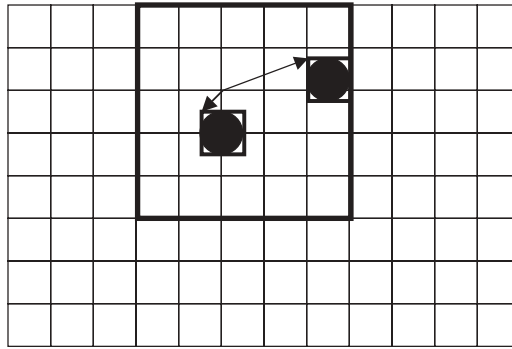


Figure 3.40 Motion estimation process for the next block in the current picture.



Prediction picture

Figure 3.41 Prediction picture from Figure 3.39 with larger search area.

in Figure 3.41. As the aim of motion-compensated prediction is to find a good match for the block to be predicted in the current picture, it does not really matter which one of these is selected. In video compression, the aim is to find a good match, not necessarily to estimate the correct motion.²

In the example just discussed, the best matching block was easily identified. In general, the task is not quite so easy as the best matching block is usually not identical to the block in the current picture. We need a measure of the similarity between the block in the current picture and the various possible prediction blocks in the prediction picture. The average difference between the blocks is of little use because large positive and negative differences might well cancel over the block. Various matching criteria have been proposed, of which the two most common are the summed absolute difference between current and candidate prediction blocks and the summed squared difference. Of these, summed absolute difference is most commonly chosen as there is no need for the relatively slow multiplication operation that is needed for squared difference, even though the use of squared difference often produces a slightly superior result. After the search operation, the block with the smallest summed absolute difference is selected as the best matching block.

Finally, we need to determine how the search is to be performed. The optimum approach (in a minimum absolute error sense assuming that total absolute error is the matching metric being used) is to compare the block we are trying to match with every block of the same size within the search area.

Consider the two 2×2 pixel block from the current picture and the search area (± 2 pixels horizontally and vertically) from the prediction picture shown in Figure 3.42. The colocated pixels in the search area are shown shaded.

There are a total of 25 possible search positions, all of which are shown in Figure 3.43. The order of the search is of no particular significance provided that all of the 25 search positions in this case are considered. Figure 3.43 shows the search

²There are other applications such as standards conversion or slow motion replay where correct motion estimation is the prime requirement.

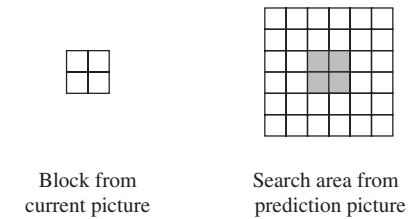


Figure 3.42 Current block and search area.

starting in the top left-hand corner of the search area with the block being moved one pixel to the right for each succeeding search until the top right-hand corner is reached. The search then reverts to the left-hand side one pixel from the top and again moves to the right one pixel at a time. This process continues until the block reaches the bottom row and concludes when the bottom right-hand corner is reached. At each search position, the summed absolute difference is calculated between the block at the search position and the block in the current picture. The block with the smallest summed absolute difference would be chosen as the motion-compensated prediction block.

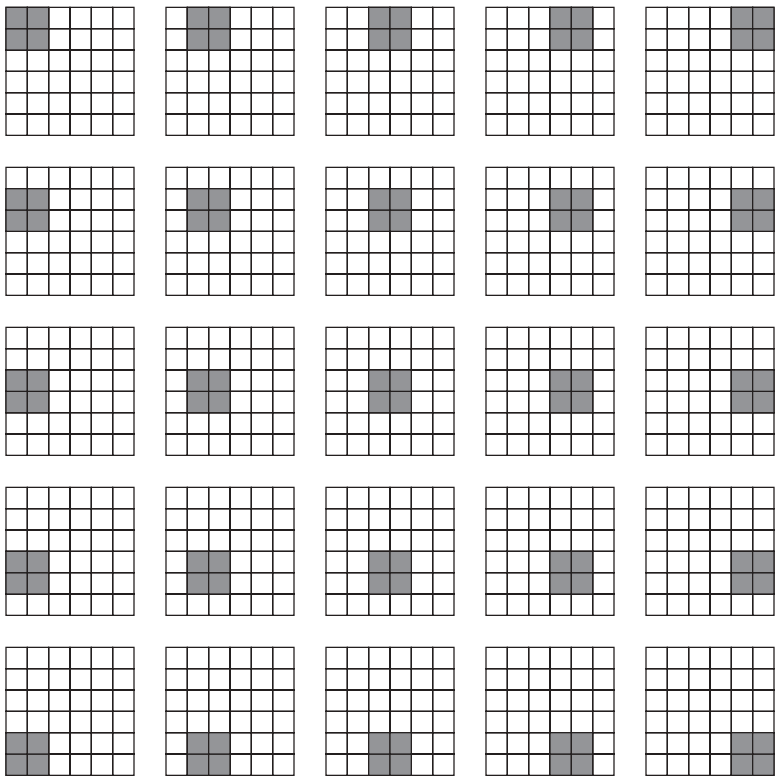


Figure 3.43 Possible search positions for the blocks shown in Figure 3.42.

When every possible search position is considered, the process is referred to as full search. For a search area of $\pm N$ pixels both horizontally and vertically, the total number of search positions to be considered is $(2N + 1) \times (2N + 1)$. For a large search area, this can be a very large number of search positions.

EXAMPLE 3.11

Block-based full search motion estimation with a search area of ± 16 pixels is used to form the prediction of the current picture from a previously coded prediction picture. Block size is 16×16 pixels. Each picture is of resolution 704×480 pixels at a picture rate of 30 pictures/s. Calculate

- the number of search positions for each block
- the number of pixel comparisons per block
- the number of pixel comparisons per picture
- the number of pixel comparisons per second.

SOLUTION The number of searches per block is $(2 \times 16 + 1)(2 \times 16 + 1) = 33 \times 33 = 1089$ searches.

For each search, two blocks of size 16×16 pixels are compared. This requires 256 pixel comparisons per search. The total number of pixel comparisons is then $1089 \times 256 = 278,874$ per block.

Each picture consists of

$$\frac{704 \times 480}{16 \times 16} = 1320 \text{ blocks.}$$

The total number of pixel operations per picture is

$$278,874 \times 1320 = 3.68 \times 10^8.$$

The picture rate is 30 pictures/s. The total number of pixel operations per second is therefore $3.68 \times 10^8 \times 30 = 11.0 \times 10^9$. ■

The computational requirements of motion estimation are large indeed. Further, in digital television applications search areas considerably greater than ± 16 pixels are regularly employed. Fortunately, special purpose digital signal processing hardware is available that is capable of working at these rates.

Reconsider the pair of pictures shown in Figure 3.31 and the blocked current picture shown in Figure 3.33. If we perform motion compensated prediction for these blocks using a search area of ± 8 pixels, then the motion-compensated prediction picture is as shown in Figure 3.44. The motion-compensated prediction difference is shown in Figure 3.45. The discontinuities in the ball, the engine of the train, and the edge of the calendar occur because within each of these blocks there is more than one form of motion—the moving foreground object (ball, engine, or calendar) and the background wallpaper. The final block chosen as the motion-compensated prediction is therefore a compromise between the two different motions. In the case of the first two blocks in the second column particularly, successful matching has

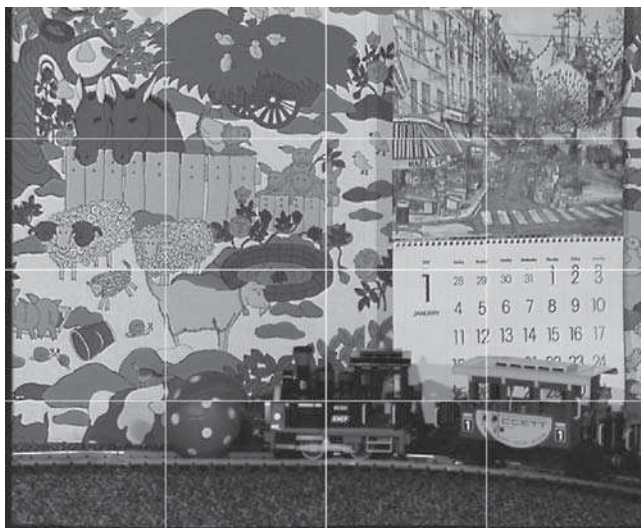


Figure 3.44 Motion-compensated prediction of picture shown in Figure 3.33(a).

been achieved because there is only one moving object (the wallpaper) in each of these blocks.

This raises the question as to what is the most appropriate block size to use. As we have already seen, the block size should be sufficiently small that the likelihood that a block contains two or more different motions is small. On the contrary, the block should be sufficiently large that the overhead associated with the motion vector is not excessive. Let us attempt to quantify this overhead. Assuming a search range

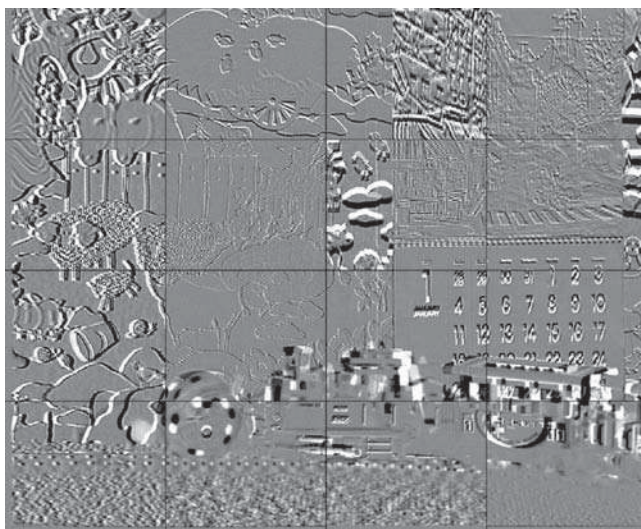


Figure 3.45 Motion-compensated prediction difference with 96×96 pixel blocks (shifted by 128 and scaled by 2).

Table 3.3 Motion vector overhead as a function of block size.

Blocks size (pixels)	Motion vector overhead (bits/pixel)
1×1	8.0000
2×2	2.0000
4×4	0.5000
8×8	0.1250
16×16	0.0313
32×32	0.0078

of ± 7 pixels, we need 4 bits to represent the horizontal component of the motion vector and 4 bits to represent the vertical component of the motion vector—a total of 8 bits/block. Table 3.3 shows the motion vector overhead for various block sizes.

As expected, the motion vector overhead is large for very small block sizes. However, by the time we reach a block size of 16×16 pixels the motion vector overhead reaches a negligibly small level. Most video coding standards use a block size of 16×16 pixels for motion-compensated prediction.

Figure 3.46(a) shows motion-compensated prediction for the pictures of Figure 3.31 using a block size of 16×16 pixels. The obvious improvement of this prediction compared to Figure 3.44 comes directly from the use of smaller blocks. There are still some minor distortions in the numbers on the calendar, and two of the spots on the ball are missing (as a result of the search area being not large enough). Of course, motion-compensated prediction can only rarely provide perfect prediction. The aim is to improve the prediction quality so that the amount of prediction difference information that needs to be transmitted is minimized. Figure 3.46(b) shows the motion vectors calculated during the motion compensation process overlayed on the motion-compensated prediction picture. This is easily done using the MATLAB *quiver* command. It can be seen that the motion estimation process has determined the true motion for much of the picture. However, there are a number of exceptions including

- The left-hand edge of the picture. The panning left of the camera causes new information to be introduced at the left-hand edge that cannot be found in the previous picture. Motion-compensated prediction simply cannot work in this circumstance.
- The white area below the large number one in the calendar. As this area is flat white, almost any motion vector would provide a good prediction and the actual vector chosen will be determined by noise within the picture. Motion-compensated prediction is still effective. A similar effect can be seen in the flat area of hay on the cart (top left of picture) and in the fence behind the sheep (middle left of picture).
- The middle of the ball. The motion estimation search area is not large enough for the correct prediction block containing the white dot to be found. Fast moving objects will invariably prove a problem for motion estimation if the



(a)



(b)

Figure 3.46 (a) Motion-compensated prediction using a block size of 16×16 pixels. (b) Motion vector overlay on motion-compensated prediction (vectors scaled by 2).

search area is not sufficiently large. Increasing the search area will improve the performance of motion estimation at the expense of an increase in computational requirements.

The motion-compensated prediction difference in this case is shown in Figure 3.47. Figure 3.48 shows the histogram for the motion-compensated difference picture shown in Figure 3.47.

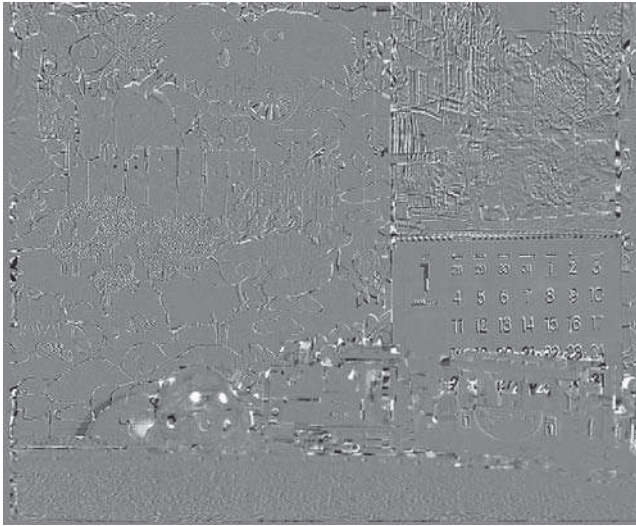


Figure 3.47 Motion-compensated prediction difference with 16×16 pixel blocks (shifted by 128 and scaled by 2).

The entropy of the motion-compensated difference picture is 5.46 bits/pixel. By comparison, the interpicture difference has an entropy of 6.18 bits/pixel and the two-dimensional intrapicture difference has an entropy of 5.79 bits/pixel. So motion compensation has reduced the entropy compared to intrapicture prediction but only by a relatively small amount.

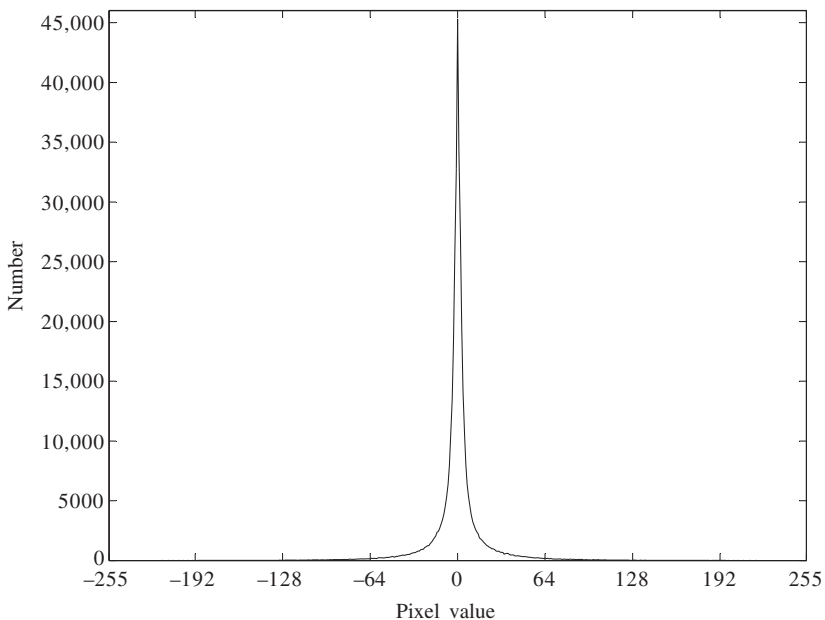


Figure 3.48 Histogram for the motion-compensated difference picture of Figure 3.47.

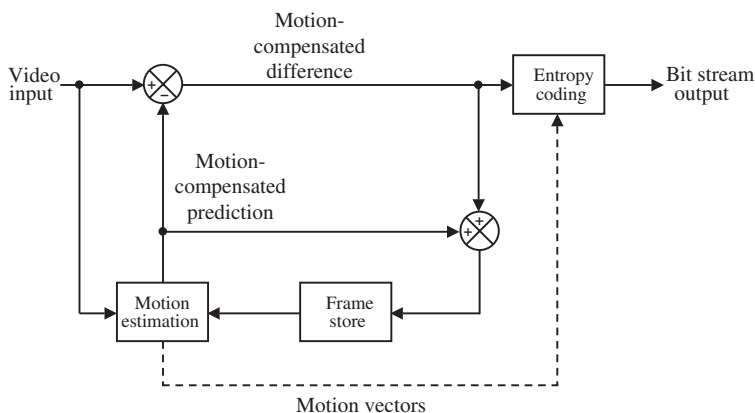


Figure 3.49 Motion-compensated encoder.

A block diagram of the motion-compensated encoder is shown in Figure 3.49 whereas the matching decoder is shown in Figure 3.50.

The decoder is simply the feedback loop of the encoder except for the entropy encoding in the coder and entropy decoding in the decoder. The motion-compensated difference signal is added to the motion-compensated prediction with the result stored in a picture store (a block of memory that holds the data for a digitized video picture).

3.3.2. Motion-Compensated Prediction to Subpixel Accuracy

Up to this point, we have assumed that motion estimation is performed to pixel accuracy. By this, we mean that each pixel in the current picture is predicted using a displaced pixel from the prediction picture. This assumes that motion occurs in multiples of single pixel displacements between pictures. This is obviously not the case in reality. Motion-compensated prediction to subpixel accuracy is possible and is supported in many video encoding standards including those used for digital television services.

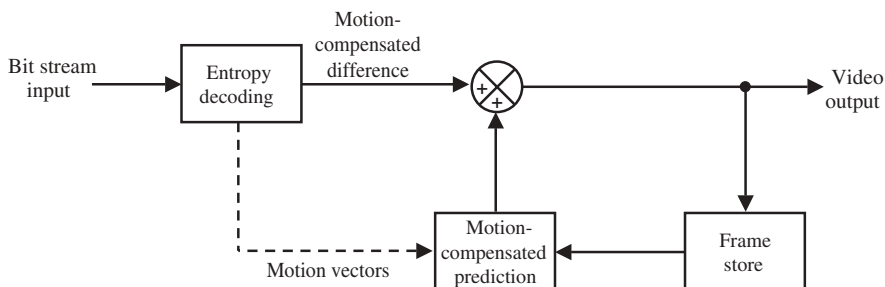


Figure 3.50 Motion-compensated decoder.

Motion-compensated prediction to subpixel accuracy can be achieved by estimating the values of pixels at subpixel displacements using bilinear interpolation from the known pixel values. This is illustrated in Example 3.12.

EXAMPLE 3.12

A 6×6 pixel block of picture data is shown in Figure 3.51. The center 4×4 pixel block is highlighted.

- Calculate the value of the 4×4 pixel block half a pixel to the right of the highlighted block.
- Calculate the value of the 4×4 pixel block half a pixel below the highlighted block.
- Calculate the value of the 4×4 pixel block half a pixel to the left and half a pixel above the highlighted block.

99	93	70	71	96	96
100	82	70	88	97	87
71	96	78	77	98	92
54	67	71	93	98	84
109	55	50	69	92	92
164	110	50	46	73	83

Figure 3.51 Picture data for Example 3.12.

SOLUTION (a) The block half a pixel to the right can be calculated by averaging each pixel in the selected region with the pixel immediately to its right. Rounding is employed to produce integer results. Half values round away from zero that is 1.5 round to 2. The final result is shown in Figure 3.52.

76	79	93	92
87	78	88	95
69	82	96	91
53	60	81	92

Figure 3.52 Half pixel shift to the right.

(b) The block half a pixel below the highlighted block can be calculated by averaging each pixel in the selected region with the pixel immediately below. Rounding is employed to produce integer results. Again, half values round away from zero. The final result is shown in Figure 3.53.

89	74	83	98
82	75	85	98
61	61	81	95
83	50	58	83

Figure 3.53 Half pixel shift down.

(c) The task of calculating the block half a pixel to the left and half a pixel up from the highlighted block is slightly more difficult. Consider the 2×2 pixel block at the top right of Figure 3.51 that is shown in Figure 3.54. The pixel half a pixel to the left and half a pixel above (X in the diagram) is calculated by averaging these four pixel values. Rounding is employed to produce integer results. Again, half values round away from zero. The final result is a pixel value of 94.

99	93
X	

Figure 3.54 Top 2×2 pixels of Figure 3.51.

When applied to the entire 4×4 pixel block, the result obtained is shown in Figure 3.55.

94	79	75	88
87	82	78	90
72	78	80	92
71	61	71	88

Figure 3.55 Half pixel shift up and left. ■

It is possible to perform motion-compensated prediction to less than half pixel accuracy. For example, it would be possible to interpolate between half pixel points to calculate quarter pixel points. However, for digital television services based on MPEG-2, motion-compensated prediction is limited to a maximum of half pixel accuracy.

3.4 QUANTIZATION

All of the techniques discussed up to this point have been lossless. This means that the decoded video signal is identical in all respects to the signal that enters the encoder. Although this ensures that the received signal is of the highest possible quality, it also ensures that the amount of compression that can be achieved is

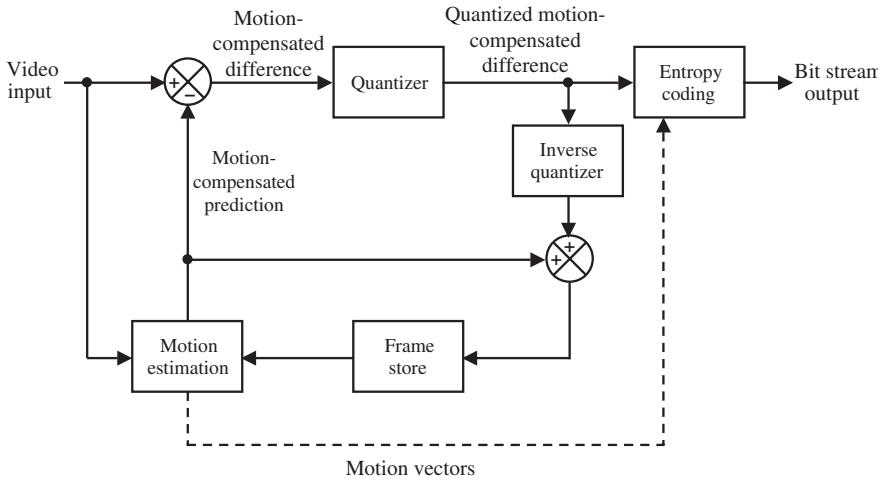


Figure 3.56 Lossy motion-compensated encoder.

limited. Fortunately, it is possible to relax this constraint and thus achieve higher compression while still maintaining more than adequate video service quality. This is achieved by removing information that is not important to the subjective quality of the video material.

While determining which information is subjectively important and which is not is quite a difficult task, the technique for removing information is more easily understood. If we revisit the lossless motion-compensated encoder shown in Figure 3.49, a lossy coder can be produced simply by introducing a quantizer to the motion-compensated difference signal as shown in Figure 3.56.

The motion-compensated difference is quantized prior to entropy coding and transmission to the decoder. As shown in Figure 3.48, the motion-compensated difference can take values in the range of ± 255 (511 possible values) although most values will lie close to zero if the motion estimation process has been effective.

Quantizer design has been extensively studied over the years. A quantizer is defined by the position of its decision and reconstruction levels. This is illustrated in the quantizer transfer function shown in Figure 3.57 for a linear quantizer where the spacing between decision levels is constant. Such a quantizer is most useful for an input signal where all values are equally likely.

The role of the quantizer is summarized in Table 3.4. Each row should be read as Lower decision level < Input < Upper decision level $\Rightarrow$ Output = Reconstruction level

For a linear quantizer, the reconstruction level is usually the average of the two decision levels that surround it (i.e., for a level defined by decision levels of +2 and +6, the reconstruction level would be +4) as this is optimum in a mean square error sense.

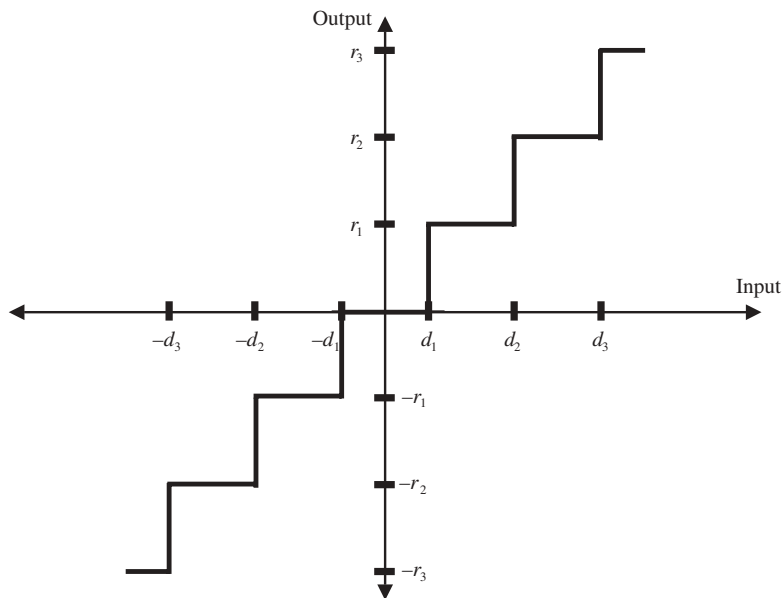


Figure 3.57 Quantizer transfer function.

Table 3.4 Decision and reconstruction levels for the quantizer of Figure 3.57.

Lower decision level	Upper decision level	Reconstruction level
$-\infty$	$-d_3$	$-r_3$
$-d_3$	$-d_2$	$-r_2$
$-d_2$	$-d_1$	$-r_1$
$-d_1$	$+d_1$	0
$+d_1$	$+d_2$	$+r_1$
$+d_2$	$+d_3$	$+r_2$
$+d_3$	$+\infty$	$+r_3$

EXAMPLE 3.13

The data shown in Figure 3.58 has been extracted from the top left-hand corner of a picture. The picture consists of gray scale values in the range 0–255.

145 162 199 155 112
149 151 153 154 162
92 85 88 187 212

Figure 3.58 Picture data for Example 3.13.

- (a) Calculate the prediction difference when one-dimensional prediction using the pixel immediately to the left of the pixel to be predicted is employed. Assume that any predictor that falls outside the picture has value 128.
- (b) The prediction difference is quantized using a linear quantizer with step size 8. Calculate the reconstructed data after prediction and quantization.

SOLUTION (a) The prediction difference is calculated by subtracting the prediction pixel from the current pixel. Remembering that the first pixel in each line is predicted by the value 128, the prediction difference is as shown in Figure 3.59.

+17	+17	+37	−44	−43
+21	+ 2	+ 2	+1	+ 8
−36	−7	+ 3	+99	+25

Figure 3.59 Unquantized prediction difference for Example 3.13.

(b) Quantization cannot be performed by simply quantizing the prediction difference shown in Figure 3.59. This is because the prediction differences shown in Figure 3.59 were formed using original pixel values. Although these values are available at the encoder, only reconstructed values are available at the decoder. For this reason, reconstructed values need to be used for prediction in the encoder as well. This means that we need to determine the reconstructed value of the current pixel before we can predict the next pixel. The solution is shown in Figure 3.60.

Current pixel	145	162	199	155	112
Prediction	128	144	160	200	152
Prediction difference	+17	+18	39	−45	−40
Quantized prediction difference	+2	+2	+5	−6	−5
Reconstructed prediction difference	+16	+16	+40	−48	−40
Reconstructed pixel	144	160	200	152	112
Current pixel	149	151	153	154	162
Prediction	128	152	152	152	152
Prediction difference	+21	−1	+1	+2	+10
Quantized prediction difference	+3	0	0	0	+1
Reconstructed prediction difference	+24	0	0	0	+8
Reconstructed pixel	152	152	152	152	160
Current pixel	92	85	88	187	212
Prediction	128	96	88	88	184
Prediction difference	−36	−11	0	+99	+28
Quantized prediction difference	−4	−1	0	+12	+3
Reconstructed prediction difference	−32	−8	0	+96	+24
Reconstructed pixel	96	88	88	184	208

Figure 3.60 Determination of reconstruction after one-dimensional prediction and quantization.

The calculations used in Figure 3.60 can be summarized as follows.

- Prediction is the reconstructed value of the previous pixel in the line.
- Prediction difference is the difference between the value of the current pixel and the prediction.
- The quantizer prediction difference is the prediction difference divided by the quantizer step size (eight in this case) rounded to the nearest integer (with a fractional part of 0.5 rounded toward zero thus 14.5 rounds to +14 and -14.5 rounds to -14). It is the quantized prediction difference that would be transmitted to the decoder.
- The reconstructed prediction difference is the quantized prediction difference multiplied by the quantizer step size (eight in this case).
- Finally the reconstructed pixel is the prediction plus the reconstructed prediction difference. This reconstructed value is used as the prediction for the next pixel. ■

The briefest perusal of Figure 3.48 indicates that the values are anything but equally likely. Quantizers can be designed to minimize the mean square error on the basis of the statistical distribution $p(x)$ of the signal to be quantized. The mean square error of a quantizer with decision levels $d_0 < d_1 < d_2 < \dots < d_L$ and reconstruction levels $r_0 < r_1 < r_2 < \dots < r_L$ is given by

$$\text{MSE} = \sum_{k=1}^L \int_{d_{k-1}}^{d_k} (x - r_k)^2 p(x) dx$$

Differentiating with respect to d_k and r_k for a fixed number of levels L leads to the following conditions for the decision and reconstruction levels.

$$d_k = \frac{r_k + r_{k+1}}{2}$$

$$r_k = \frac{\int_{d_{k-1}}^{d_k} xp(x) dx}{\int_{d_{k-1}}^{d_k} p(x) dx}$$

For the predictive encoders considered up to this point, $p(x)$ can be approximated by a Laplacian distribution with mean zero and variance $2/\lambda^2$.

$$p(x) = \frac{\lambda}{2} e^{-\lambda|x|}$$

The prediction difference is highly peaked around zero. This results in the quantizer steps being closely spaced near zero prediction difference and getting larger as the prediction difference gets larger. Such a nonlinear quantizer is shown in Figure 3.61.

Although quantization is an important aspect of video compression, in modern coding standard quantization is not performed on prediction differences. Rather, it is transform coefficients that are quantized. We therefore leave any further

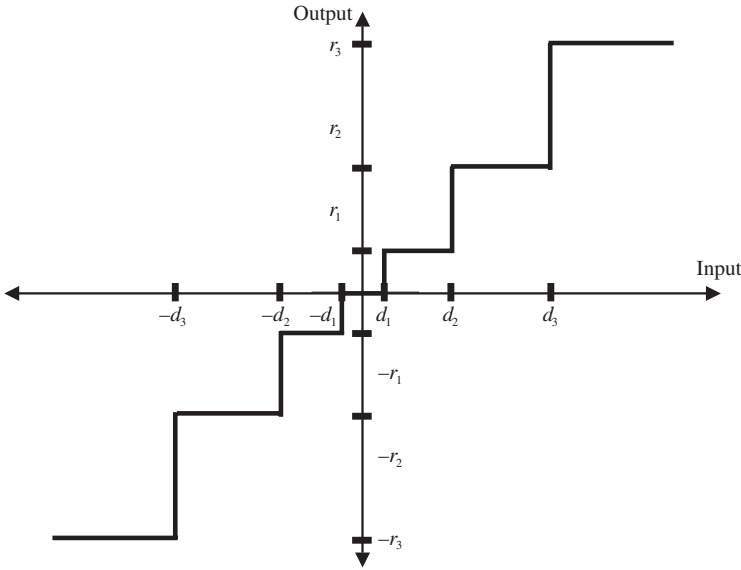


Figure 3.61 Nonlinear quantizer.

consideration of quantization until after transform coding has been introduced in Chapter 4.

3.5. RATE-DISTORTION CURVES

The introduction of quantization allows the data rate produced by the encoder to be traded against the quality of the reconstructed service. A small value of the quantizer leads to a high-quality reconstruction at a high data rate. A large value of the quantizer greatly reduces the data rate, but produces a much lower quality reconstruction.

A common method of measuring the quality of a reconstructed video service is the peak signal-to-noise ratio (PSNR). The mean square error between an original and a reconstructed picture of size $M \times N$ pixels is defined as

$$\text{MSE} = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N (x_{i,j} - \hat{x}_{i,j})^2$$

where $x_{i,j}$ is the value of the original pixel and $\hat{x}_{i,j}$ is the value of the reconstructed pixel.

The PSNR is then defined as

$$\text{PSNR} = 10 \log_{10} \frac{(\text{Peak-to-peak signal})^2}{\text{MSE}}$$

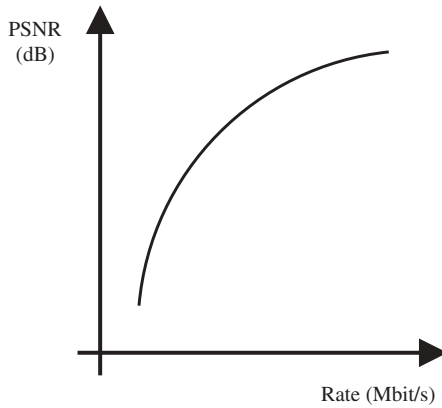


Figure 3.62 Typical rate-distortion curve.

For a 256 level gray scale image the peak to peak signals is 255 and so

$$\text{PSNR} = 10 \log_{10} \frac{(255)^2}{\text{MSE}}$$

Plotting PSNR versus the generated data rate produces a curve called the rate-distortion curve. Rate-distortion curves can be useful in the comparison of the performance of various coding techniques on the same video material. An example of a rate-distortion curve, which has been drawn based on the distortion at a number of data rates, is shown in Figure 3.62.

3.6. SUMMARY

In this chapter we have introduced the idea of predictive encoding to reduce the amount of data required to represent a picture. We have seen that prediction can be performed in the spatial domain (intrapicture prediction) or between pictures separated in time (interpicture prediction). The prediction difference is then further coded using a Huffman code to exploit the fact that not all values of prediction difference are equally likely.

The chapter has also introduced the concept of motion-compensated prediction that attempts to compensate for the movement of objects between pictures prior to prediction. This was shown to significantly enhance the performance of interpicture prediction.

Finally, the concept of quantization was introduced. This lossy technique allows the prediction difference to be represented more coarsely. Distortion is introduced while at the same time a major saving in the required data rate is achieved. Distortion and rate can be traded off by varying the value of the quantizer step size.

Motion-compensated prediction is an important technique in the coding of digital television services. However, rather than transmitting the quantized prediction

difference, another technique known as transform coding is employed prior to quantization. Transform coding is the topic of the next chapter.

PROBLEMS

- 3.1** Explain briefly the advantages and disadvantages of variable length codes.
- 3.2** An encoder produces an output consisting of four different symbols (a to d) having the probabilities of appearance given in Table 3.5.

Table 3.5 Probabilities for Problem 3.2.

Symbol	a	b	c	d
Probability	0.20	0.30	0.40	0.10

- (a) What is the entropy of encoder output?
- (b) If fixed length code words are used, how many bits would be required to represent each symbol? What is the coding efficiency?
- (c) The output symbols are encoded with a variable length Huffman code. What is the average number of bits per symbol needed to transmit the symbols? What is the coding efficiency?
- 3.3** An encoder produces an output consisting of 10 different symbols (a to j) having the probabilities of occurrence given in Table 3.6.

Table 3.6 Probabilities for Problem 3.3.

Symbol	a	b	c	d	e	f	g	h	i	j
Probability	0.02	0.05	0.08	0.10	0.25	0.20	0.18	0.07	0.04	0.01

- (a) What is the entropy of encoder output?
- (b) If fixed length code word are used, how many bits would be required to represent each symbol? What is the coding efficiency?
- (c) The output symbols are encoded with a variable length Huffman code. What is the average number of bits per symbol needed to transmit the symbols? What is the coding efficiency?
- 3.4** An encoder has been designed that produces three output symbols namely go up (U), no change (N), or go down (D). The relative probabilities of each symbol are given in Table 3.7.

Table 3.7 Probabilities for Problem 3.4.

Symbol	U	N	D
Probability	0.05	0.90	0.05

- (a) Determine the coding efficiency if these symbols are transmitted individually using
 - (i) a fixed word length coding scheme
 - (ii) a variable word length Huffman code.
 - (b) Repeat part (a) after the three symbols have been grouped in pairs (i.e., UU, UN, UD, NU, NN, ND, DU, DN, DD) prior to encoding. You may assume the symbols are independent, that is $P(AB) = P(A)P(B)$.
 - (c) Repeat part (a) after the three symbols have been grouped in triplets.
 - (d) Comment on the results obtained in terms of the suitability of Huffman codes in cases where one symbol is much more likely than any of the others.
- 3.5** An encoder produces a binary output (i.e., a 0 or a 1). The probability of a 0, P_0 , is given by p whereas the probability of a 1, P_1 , is given by $(1-p)$.
- (a) Plot the information content of the encoder output in bits per symbol as a function of p .
 - (b) If each output symbol is represented by a fixed length code word, plot the encoder output in bits per symbol as a function of p .
 - (c) If output symbols are grouped in pairs and then variable length coded, plot the encoder output in bits per symbol as a function of p .
 - (d) If output symbols are grouped in triplets and then variable length coded, plot the encoder output in bits per symbol as a function of p .
 - (e) Comment on the range of values of p where each of these approaches is appropriate. Give reasons for your decision.
- 3.6** Five symbols (a to e) are encoded using the variable length codes shown in Table 3.8.

Table 3.8 Variable length code words for Problem 3.7.

Symbol	code word
a	0
b	10
c	110
d	1110
e	1111

- (a) Determine the bit stream when the transmitted string of symbols is as shown in Figure 3.63.

b a a c a d a e a a **Figure 3.63** Symbol stream for Problem 3.7.

- (b) A single transmission results in an error in the reception of the first bit of the bit stream. Determine the symbol stream that will be output by the decoder.
 - (c) Repeat (b) for the case where the transmission error occurs in the second bit of the bit stream.
 - (d) Comment briefly on the impact of transmission errors on information coded with variable length code words.
- 3.7** A source encoder produces strings of zero bits of various length followed by the value one. The output of the source encoder is to be transmitted as a series of variable length code words that define the length of the string of zeros that occur before each one. The variable length code words are defined in Table 3.9.

Table 3.9 Variable length code words for Problem 3.7.

Length of string of zeros	Source encoder output	code word
0	1	1100
1	01	100
2	001	01
3	0001	00
4	00001	101
5	000001	1101
6	0000001	1110

Use this scheme to encode the sample encoder output shown in Figure 3.64.

001100100000010000101100001000100000100100100010010001

Figure 3.64 Sample encoder output for Problem 3.7.

3.8 The run length coding scheme described in Problem 3.7 is to be extended so that it can handle a string of zeros up to a length of 63. Rather than define a new Huffman code word for each case, the scheme defined in Table 3.9 is modified by including an additional code word that covers any string of zeros greater than six.

The new code word consists of the prefix 1111 (which is not an existing valid code word) followed by the number of zeros as a 6-bit number. Thus a string of ten zeros followed by a one would be represented by the code word 1111001010. Use the modified coding scheme to encode the sample encoder output shown in Figure 3.65.

001100100000000100001011000000000000010001000001001001000100100000001

Figure 3.65 Sample encoder output for Problem 3.8.

3.9 Figure 3.66 shows a block of data from a picture containing pixels with values in the range 0–255.

62 61 45 25

28 32 30 27

21 20 22 20

17 16 18 18

Figure 3.66 Picture data for Problem 3.9.

- (a) Calculate the prediction difference after one-dimensional predictive encoding using the value of the pixel to the left of the pixel to be predicted as the prediction. Assume that all pixels surrounding the pixels to be predicted have value 128.
- (b) Calculate the prediction difference after two-dimensional predictive encoding using the average of the value of the pixel to the left of the pixel to be predicted and the value of the pixel directly above the pixel to be predicted as the prediction. Assume that all pixels surrounding the pixels to be predicted have value 128. Fractional values should be rounded toward zero.

- 3.10** Consider the picture data given in Figure 3.66. In this case, the prediction difference is to be quantized by a linear quantizer of the type shown in Figure 3.57. The quantizer step size is 4 (i.e., decision levels are at $\pm 2, \pm 6, \pm 10$, etc.) and the reconstruction level is at the mid-point of the relevant decision levels. Calculate the reconstructed pixel values that would be output from the decoder in these circumstances.
- 3.11** Repeat Problem 3.10 using a variety of quantizer step sizes in the range 1–256. Plot the mean square error introduced by quantization versus the quantizer step size. Comment on the result.
- 3.12** Figure 3.67 shows a block of data from a picture containing pixels with values in the range 0–255.

60	121	222	247	97	11	48	66
105	227	240	172	48	16	62	72
74	186	246	206	169	41	25	69
184	254	191	212	131	12	58	68
115	241	225	187	215	96	17	71
236	227	186	210	189	37	35	81
193	242	202	196	220	157	20	42
240	207	206	201	207	79	14	78

Figure 3.67 Picture data for Problem 3.12.

- (c) Calculate the prediction difference after one-dimensional predictive encoding using the value of the pixel to the left of the pixel to be predicted as the prediction. Assume that all pixels surrounding the pixels to be predicted have value 128.
- (d) Calculate the prediction difference after two-dimensional predictive encoding using the average of the value of the pixel to the left of the pixel to be predicted and the value of the pixel directly above the pixel to be predicted as the prediction. Assume that all pixels surrounding the pixels to be predicted have value 128. Fractional values should be rounded toward zero.
- 3.13** Consider the picture data given in Figure 3.67. In this case, the prediction difference is to be quantized by a linear quantizer of the type shown in Figure 3.57. The quantizer step size is 8 (i.e., decision levels are at $\pm 4, \pm 12, \pm 20$, etc.) and the reconstruction level is at the mid-point of the relevant decision levels. Calculate the reconstructed pixel values that would be output from the decoder in these circumstances.
- 3.14** Repeat Problem 3.13 using
- a quantizer step size of 1
 - a quantizer step size of 128
 - Comment on the results achieved.
- 3.15** A video encoder designed for digital television operates on pictures of resolution 720×576 pixels at a frame rate of 25 pictures/s. Full search motion estimation is employed

using 16×16 pixel blocks with minimum absolute difference as the matching criteria. The search area employed is ± 64 pixels.

- (a) Calculate the total number of search positions and the total number of absolute pixel difference operations required each second in the encoder.
 - (b) Calculate the number of search positions and the total number of absolute pixel difference operations required each second in the decoder.
- 3.16** A video encoder designed for high-definition digital television operates on pictures of resolution 1440×480 pixels at a rate of 60 pictures/s. Full search motion estimation is employed using 16×16 pixel blocks with minimum absolute difference as the matching criteria. The search area employed is ± 128 pixels.
- (a) Calculate the total number of search positions and the total number of absolute pixel difference operations required each second in the encoder.
 - (b) Calculate the number of search positions and the total number of absolute pixel difference operations required each second in the decoder.
- 3.17** The enormous number of computations required in the motion estimation process can be achieved in practice, but requires significant use of parallel processing (i.e., several calculations being performed at the same time) to be successfully achieved. What parts of the motion estimation process can occur in parallel. What is the speed-up factor achieved by each type of parallel processing?
- 3.18** In what circumstances would you expect motion-compensated prediction to perform poorly? Give real world examples in support of your answer.
- 3.19** Consider the block of pixel data shown in Figure 3.68. The block of interest is surrounded by a black border.
- (a) Calculate the block half a pixel to the left block of interest.

50	61	73	75	81	84	84	91	91	82
78	110	106	100	100	102	100	106	100	101
42	102	121	102	98	97	96	98	95	93
52	108	103	96	93	91	94	88	89	92
46	80	111	96	95	89	95	81	71	94
76	104	98	92	93	91	89	48	58	103
79	92	105	97	90	92	93	72	26	72
97	98	99	95	90	96	90	49	59	106
98	93	102	100	91	92	89	77	61	87
91	96	106	90	90	94	85	66	85	101

Figure 3.68 Pixel data for Problem 3.19.

- (b) Calculate the block half a pixel up from the block of interest.
 - (c) Calculate the block half a pixel to the right and half a pixel down from the block of interest.
- 3.20** How would you go about performing motion estimation to quarter pixel accuracy? Reconsider the pixel data given in Figure 3.68.
- (a) Calculate the block one quarter of a pixel to the left block of interest.
 - (b) Calculate the block three quarters of a pixel up from the block of interest.
 - (c) Calculate the block three quarters of a pixel to the right and one quarter of a pixel down from the block of interest.
- 3.21** When a perfect match is found during the motion estimation process, there is no need to perform further searches. However, most hardware implementations of motion estimation algorithms would continue the search in any case. Why do you think this would be the case?
- 3.22** Most video encoders that incorporate motion-compensated prediction allow poorly predicted blocks to be coded in intrapicture mode (i.e., without any form of temporal prediction). What impact would this have on the bit stream transmitted between the encoder and the decoder?

MATLAB EXERCISE 3.1: HUFFMAN CODING

Huffman encoding is commonly employed in modern digital video encoders to reduce the amount of information needed to represent digital information. In this exercise you will develop a MATLAB program that calculates the Huffman code for an original picture.

Section 1 Design of Huffman code words

Write a MATLAB program to calculate the Huffman code words for a video picture. Hence calculate the average number of bits per pixel required to represent a number of different pictures.

In order to perform this task, you will need to complete the following steps:

- Calculate the probability of each pixel value in the picture.
- Develop the Huffman tree. This requires the development of a representation of the linkages within the computer. One way of doing this is to track where each element in column i of the tree maps to in column $i+1$. It is then possible to read the tree from left to right. Reversing the code word read gives the Huffman code word.
- Calculate the average number of bits per pixel by multiplying the probability of each pixel value by the number of bits in the code word used to represent it and then summing over all possible pixel values. Note that not all 256 possible pixel values may occur in a particular picture.

Section 2 Huffman code efficiency

Write a MATLAB program to calculate the entropy of a picture or a prediction difference picture. Use your routine to calculate the entropy of the various pictures

studied in Section 1. Hence calculate the coding efficiency of the Huffman code for each picture.

Section 3 Huffman code effectiveness

Plot in the same figure the number of bits allocated by your Huffman algorithm to each pixel brightness and the number of bits suggested by the entropy value for that pixel. Comment on the result. Note that not all pixel values are used in some pictures so care will be needed in calculating the individual pixel entropies.

MATLAB Hint: The MATLAB command

hold on

allows more than one graph to be plotted on the same set of axes. The hold is turned off by the

hold off

command

MATLAB EXERCISE 3.2: DIFFERENTIAL PULSE CODE MODULATION

In this exercise you will explore the bit-rate savings that can be achieved using one-dimensional and two-dimensional lossless predictive encoding. Linear quantization is then incorporated in Section 2, thus allowing a study in the trade-offs possible between bit rate and picture quality in differential pulse code modulation.

Section 1 Lossless intrapicture predictive encoding

For at least one picture taken from different video sequences calculate

- the entropy of the horizontal one-dimensional prediction difference (i.e., using the pixel directly to the left of the pixel to be predicted as the prediction).
- the entropy of the vertical one-dimensional prediction difference (i.e., using the pixel directly above the pixel to be predicted as the prediction).
- the entropy of the two-dimensional prediction difference (i.e., using the average of the pixel directly to the left of the pixel to be predicted and the pixel directly above the pixel to be predicted as the prediction).

Comment on these results.

Section 2 Lossy intrapicture predictive encoding

Section 1 involves the lossless encoding of a picture since the original picture can be reconstructed at the decoder. The process is made lossy by the introduction of quantization. Extend the predictive encoders developed in Section 1 to include linear

quantization of the type shown in Figure 3.57. For a quantizer with step size 4, the decision levels would be $\pm 2, \pm 6, \pm 10, \dots$ and the reconstruction levels $0, \pm 4, \pm 8, \dots$

Use your simulation to calculate the PSNR in decibels of the reconstructed picture and the entropy of the quantized prediction difference signal for a range of quantizer step sizes. Plot PSNR versus entropy for each of the three prediction techniques. Such a plot is called a rate-distortion curve. Also examine the quality of the reconstructed picture as the quantizer step size is increased. Summarize your results.

Note: A quick check that your program is operating correctly is to use a quantizer step size of 1. In this case, the PSNR should be infinite and the entropy the same as in Section 1.

MATLAB EXERCISE 3.3: TEMPORAL PREDICTION AND MOTION ESTIMATION

In this exercise we study the effectiveness of temporal prediction techniques. This begins with a study of interpicture prediction (i.e., predicting the current pixel using the pixel in the same location in the previous picture). We then move on to look at motion-compensated prediction. Quantization is incorporated into both techniques.

Section 1 Lossless and lossy interpicture prediction

For several video sequences:

- Calculate the entropy of the interpicture prediction difference (i.e., using the pixel in the same location as the current pixel in the previous picture as the predictor). Naturally picture $N+1$ will be predicted by picture N in this experiment.
- Take the fifth picture in each sequence and perform interpicture prediction using each of the pictures that precedes it (i.e., pictures 1–4). Comment on the effectiveness of interpicture prediction as the spacing between the pictures increases.
- The previous two parts deal with the lossless interpicture encoding of a picture because the original picture can be reconstructed at the decoder. The process is made lossy by the introduction of quantization. Extend the predictive encoders developed to include linear quantization of the type shown in Figure 3.57. For a quantizer with step size 4, the decision levels would be $\pm 2, \pm 6, \pm 10, \dots$ and the reconstruction levels $0, \pm 4, \pm 8, \dots$
- Use your simulation to calculate the peak signal-to-noise ratio in decibels of the reconstructed picture and the entropy of the quantized prediction difference signal for a range of quantizer step sizes. Plot PSNR versus entropy for interpicture prediction. Such a plot is called a rate-distortion curve. Also examine the quality of the reconstructed picture as the quantizer step size is increased. You should note that the picture quality looks good when a small quantizer is used, gets worse as the quantizer is increased and then starts to improve again when the quantizer step size becomes large. Explain why this

is the case and why this improvement at large quantizer step sizes is of no value in real applications.

Note: A quick check that your program is operating correctly is to use a quantizer step size of 1. In this case, the PSNR should be infinite and the entropy the same as in the lossless case.

Section 2 Lossless motion-compensated prediction

For several video sequences

- Calculate the entropy of the motion-compensated prediction difference using a block size for motion estimation of 16×16 pixels and a search range of ± 8 pixels both vertically and horizontally. Use minimum absolute error as the matching criteria for all of the motion estimation experiments contained in this exercise. Do not forget to include the overhead associated with the motion vectors in this calculation. Naturally picture $N+1$ will be predicted by picture N in this experiment. Make use of the MATLAB quiver command to overlay the motion vectors onto the picture being predicted.
- For a fixed search area of ± 8 pixels both vertically and horizontally, perform experiments to determine the effect of changing the block size used for motion estimation on the total entropy of the motion-compensated prediction difference. Make sure that you include the overhead associated with the motion vectors.
- For a fixed block size of 16×16 pixels, perform experiments to determine the effect of changing the search area to be used for motion estimation on the entropy of the motion-compensated prediction difference. Make sure that you include the overhead associated with the motion vectors.
- Find a pair of pictures from a video sequence that are on either side of a scene cut (or alternatively use two pictures from different video sequences) and calculate the entropy of the motion-compensated prediction difference. This situation occurs at every scene cut in a sequence. Suggest methods by which this limitation of motion-compensated prediction could be overcome.
- Take the fifth picture in each video sequence and perform motion-compensated prediction using each of the pictures that precedes it (i.e., pictures 1–4). Comment on the effectiveness of motion-compensated prediction as the spacing between the pictures increases.

Section 3 Lossy motion-compensated prediction

The previous section dealt with the lossless motion-compensated prediction of a picture because the original picture can be reconstructed at the decoder. The process is made lossy by the introduction of quantization. Extend the motion-compensated encoders developed in Section 2 to include linear quantization of the type shown in Figure 3.57. For a quantizer with step size 4, the decision levels would be ± 2 , ± 6 , ± 10 , ... and the reconstruction levels 0, ± 4 , ± 8 , ...

Use your simulation to calculate the PSNR in decibels of the reconstructed picture and the entropy of the quantized motion-compensated prediction difference

signal (including motion vector overhead) for a range of quantizer step sizes. Plot the rate-distortion curve. Also examine the quality of the reconstructed picture as the quantizer step size is increased. Compare the results obtained with those for lossy interpicture prediction calculated in Section 1.

MATLAB EXERCISE 3.4: FAST SEARCH MOTION ESTIMATION

Full search motion estimation always provides optimum performance (in terms of the matching criteria used at least). However, for a search area of $\pm M$ horizontally and $\pm N$ vertically, the total number of search required is $(2M + 1) \times (2N + 1)$ which can be very large. A number of fast search motion estimation techniques have been proposed by various researchers. Invariably, the aim is to search only a subset of the possible search positions and to choose the search positions checked in an intelligent way that increases the chances of obtaining the best (or at least a good) match. In this exercise we will study one of these techniques and compare its performance to full search motion estimation.

The fast search technique to be studied is three-step search motion estimation that is designed to work with a search area of ± 7 pixels both horizontally and vertically. Figure 3.69 shows a 2×2 pixel block shaded surrounded by a search area of ± 7 pixels.

The initial nine searches in the prediction picture are at the collocated block and at blocks ± 4 pixels above, ± 4 pixels below, and a combination of both of these displacements as shown in Figure 3.70. This is a total of nine searches as shown in the figure although the order of search is not important. One of these yields the best match in terms of the measure being used (say absolute error). In this case, we assume that this occurs at search position 1 that is shown in a different color in Figure 3.70.

Position 1 becomes the center of the next search and the new search positions are located at blocks ± 2 pixels above, ± 2 pixels below, and a combination of both of these displacements as shown in Figure 3.71. This requires eight new searches (searches 10–17 although again the search order is not significant). One of these nine

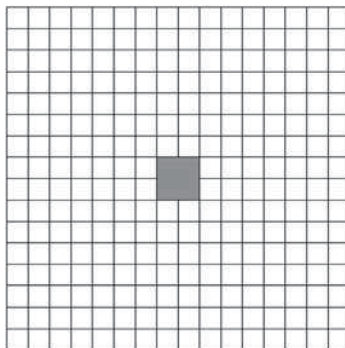


Figure 3.69 2×2 pixel block surrounded by a ± 7 pixel search area in the current picture.

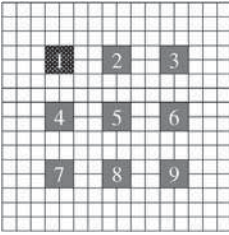


Figure 3.70 First step of three-step search.

positions (the best match could still be at position 1) is the best match in terms of the matching criteria. In this case, we will assume that this occurs at search position 14 that is shown in a different color in Figure 3.71.

Position 14 becomes the center of the next search, and the new search positions are located at blocks ± 1 pixels above, ± 1 pixels below, and a combination of both of these displacements. Because there is significant overlap between the search positions, no attempt will be made to draw them—hopefully the process is now sufficiently clear that this is not necessary. This again requires 8 new searches (searches 18–25 and again the search order is not significant). One of these nine positions (the best match could still be at position 14) will be the best match in terms of the matching criteria. This match will be taken as the best matching block and its displacement compared to the collocated is the motion vector.

The technique works on the basis that the closer one moves to the best match block, the smaller is the matching error. If this were always the case (which, of course, it is not), the three-step search algorithm would always produce the best matching block. In the remainder of this exercise we will explore the success of this fast search method.

Section 1 Full search motion estimation

- For a range of pairs of adjacent pictures extracted from different video sequences, calculate the entropy of the motion-compensated prediction difference using a block size for motion estimation of 16×16 pixels and a search range of ± 7 pixels both vertically and horizontally. Use minimum absolute error as the matching criteria for all of the motion estimation experiments contained in this exercise. Do not forget to include the overhead associated with the motion vectors in this calculation. Naturally picture

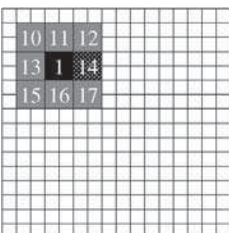


Figure 3.71 Second step in three-step search.

$N+1$ will be predicted by picture N in this experiment. Make use of the MATLAB quiver command to overlay the motion vectors onto the picture being predicted.

Section 2 Fast search motion estimation

- Repeat the previous calculation using the three-step search fast motion estimation algorithm. Comment on the performance in terms of the entropy of the motion-compensated prediction difference, the accuracy of the motion estimation, and the amount of computation needed.
- Take the fifth picture in each sequence and perform motion-compensated prediction using each of the pictures that precedes it (i.e., pictures 1–4) using both full search and the fast search technique. Comment on the effectiveness of motion-compensated prediction techniques as the spacing between the pictures increases.
- Extend the three-step search to cover a search area of ± 15 pixels by including a fourth step in the search process. Again compare the performance of the fast search algorithm with full search in terms of the entropy of the motion-compensated prediction difference, the accuracy of the motion estimation, and the amount of computation needed.